Syntax of CSL

Contents

This document describes the basic structures of the CSL language.

Type system overview

The basic types of CSL are:

- void type (`void`)
- signed integers (`i8`, `i16`, `i32`, `i64`)
- unsigned integers (`u8`, `u16`, `u32`, `u64`)
- floating point numbers (`f16`, `f32`, `bf16`, `cb16`)

Arrays types are spelled `[num_elements] base_type`, for example: `[3] i16`. Array literals are specified by an array type followed by a list of values, for example: `[3]i16 {1, 2, 3}`

For a detailed introduction to the type system of CSL, see Type System in CSL.

Variables

Variable declarations are composed of a mutability specifier, a name, a type and an initializer:

```
const ten_i : i16 = 10;
var   ten_f : f16 = 10.0;
param ten_d : f32 = 10.0;
```

A `const` or `param` variable cannot have its value changed after it has been initialized, whereas a `var` variable has no such restriction.

The initializer expression is:

- Mandatory for `const` variables.
- Optional for `var` variables.
- Optional for `param` variables. If one is not provided, the `param` must be initialized through the module import system. See Modules.

The type expression is optional. If one is not provided, the initializer expression is mandatory and it is used to deduce the type of the variable:

```
const ten_a : i16 = 10;
const ten_b       = ten_a;  // ten_b is also i16.
param my_param1; // ok, the initializer is provided later.
```

Variable declarations may optionally have an alignment requirement:

```
const aligned_var1 : i16 align(32) = 10;
const aligned_var2       align(64) = ten_a;
```

The memory address of the corresponding variable is guaranteed to have *at least* the specified alignment. Alignment is specified as a number of bytes and must always be a power of two.

Global variables can be used before their declaration. For example, the following is legal:

```
fn my_fn(x: f16) void {
  my_global = x;
}

var my_global : f16;
```

Global variable declarations may also optionally specify the name of the link section:

```
var global_var1 : i16 linksection(".mySection") = 10;
```

By specifying the link section name `.mySection`, the global variable gets placed into a separate object file section named `.mySection`, instead of being placed into the object file section with the rest of the global variables.

The `linksection` attribute can be used together with the compiler flag `--link-section-start-address-bytes` to place global variables at particular memory addresses:

```
var section1: u16 linksection(".mySection1") = 0xabcd;
var section2: u16 linksection(".mySection2") = 0x1234;

// $ cslc-driver ... \
//   --link-section-start-address-bytes=".mySection1:40960,.mySection2:40980"
```

In the example above, the variable `section1` is placed at the memory address 40960 (bytes), and `section2` is placed at 40980.

Global variable declarations may also optionally specify the name of the ELF symbol corresponding to the variable:

```
var global_var : i16 linkname("different_name") = 10;
```

In this example, the global variable known as `global_var` within CSL gets assigned the name `different_name` in the compiled object file. This can be useful to control the name of symbols that are intended to be referenced by other object files as external data. Any comptime expression evaluating to a value of type `comptime_string` may be used for `linkname`.

Global variable declarations may optionally specify a storage class (either `export` or `extern`). If a variable is declared `export`, it is made accessible to other separately-compiled objects, and is guaranteed not to be eliminated from the compiled object. If a variable is declared `extern`, it is assumed that its definition will be supplied by another object that will later be linked with the object we are compiling. An `extern` declaration must *not* initialize the variable.

Variables with the `export` or `extern` storage classes must have an *export-compatible* type. See Storage Classes for details.

```
// Variable 'x' will be available to other objects that are linked with
// this program.
export var x: i16 = 12;

// We expect that variable 'y' will be provided by another object that is
// to be linked with this program.
extern var y: i16;

// Variable 'foo' will be available under the name 'alias_for_foo' to other
// objects that are linked with this program.
export var foo: i16 linkname("alias_for_foo") = 42;

// Variable 'alias_for_bar' will be aliased to the a variable 'bar' provided
// by another object that is to be linked with this program.
extern var alias_for_bar: i16 linkname("bar");
```

Pointers

To obtain a pointer to a variable, the address-of operator `&` is used:

```
var x = [2]i16 {0, 1};
var ptr = &x; // ptr is a *[2]i16

const y = [2]i16 {0, 1};
const const_ptr = &y; // const_ptr is a *const[2]i16
```

Only variables are addressable, as such it is illegal to obtain the address of a temporary:

```
const x = [2]i16 {0,0};
const ok_ptr = &x[1];

const bad_ptr = &(([2]i16 {0,0})[1]); // compile-time error.
```

To dereference a pointer, the dereference operator `.*` is used:

```
var x = [2]i16 {0, 1};
var ptr_to_x = &x; // ptr is a *[2]i16

var copy_of_x = ptr_to_x.*; // copy_of_x is a [2]i16

var element_of_x = ptr_to_x.*[1]; // element_of_x is an i16
```

Functions

Function definitions require a `fn` or `task` keyword, a name, an optional sequence of parameters, a return type and a function body:

```
fn foo(arg : i16) i32 { ... }
task my_task(arg : i16) void { ... }
```

Functions that are declared using the `task` keyword are called *tasks*.

All function parameters are implicitly `const` variables.

It is unspecified whether function parameters are passed by value or by reference. If it is necessary to modify a function argument, the function parameter must be declared with a pointer type:

```
fn foo(arg : *i16) void {
  arg.* = 42;
}

fn bar() void {
  var x : i16 = 0;
  foo(&x); // x is now 42.
}
```

The type of a function parameter may be specified with the keyword `anytype`. In this case, the compiler will create a specialized copy of the function based on the *type* of the corresponding argument used at the call site. This is similar to `typename` templates in C++.

```
/// Computes base ^ exp
fn pow(base : anytype, exp : @type_of(base)) @type_of(base) {
  const base_type = @type_of(base);
  if (base_type == i16) {
    // ... integer implementation ...
  }
  if (base_type == f16) {
    // ... float implementation ...
  }
  return @as(base_type, 0);
}

task t() void {
  const v1 : i16 = ...;
  pow(v1, 6); // specialized for `i16`.

  const v2 : f16 = ...;
  pow(v2, 6.0); // specialized for `f16`.
}
```

Function parameters can optionally be marked with the `comptime` keyword (see Comptime). In this case, the compiler will create a specialized copy of the function based on the *value* of the corresponding argument at the call site. The argument must be comptime-known. This is similar to non-type template parameters in C++.

```
/// This function is specialized for each value of base_type.
fn copy(size : i16, comptime base_type : type,
        dest : [*]base_type, src : [*]base_type) void {

  for (@range(i16, size)) |idx| {
    dest[idx] = src[idx];
  }
}

task t() void {
  var src = @constants([10]i16, 42);
  var dest : [10]i16;
  copy(10, i16, &src, &dest); // specialized for i16.
}
```

Functions may also optionally specify the name of their link section:

```
fn foo () linksection(".mySection") void { ... }
```

By specifying the link section name `.mySection`, the function gets placed into a separate object file section named `.mySection`, instead of being placed into the object file section with the rest of the functions.

Tasks may not specify a link section name.

Function definitions may also optionally specify the name of the ELF symbol corresponding to the function:

```
fn foo () linkname("bar") void { ... }
```

In this example, the function known as `foo` within CSL gets assigned the name `bar` in the compiled object file. This can be useful to control the name of functions that are intended to be called by other object files as `extern` functions. Any comptime expression evaluating to a value of type `comptime_string` may be used for `linkname`.

Function declarations may optionally specify a storage class (either `export` or `extern`). If a function is declared `export`, it is made accessible to other separately-compiled objects, and its definition is guaranteed not to be eliminated from the compiled object. If a function is declared `extern`, it is assumed that its definition will be supplied by another object that will later be linked with the object we are compiling. An `extern` function declaration must *not* contain a function body.

Functions with the `export` or `extern` storage classes must have an *export-compatible* type. See Storage Classes for details.

```
// Function 'f' will be available to other objects that are linked with
// this program.
export fn f(x: i16, y: i16) { return x+y; }

// We expect that function 'g' will be provided by another object that is
// to be linked with this program.
extern fn g(f16, f16) f16;

// Function 'foo' will be available under the name 'alias_for_foo' to other
// objects that are linked with this program.
export fn foo(x: *i16) i16 linkname("alias_for_foo") { return x.*; }

// Function 'alias_for_bar' will be aliased to the a function 'bar' provided
// by another object that is to be linked with this program.
extern fn alias_for_bar(*f16) f16 linkname("bar");
```

inline fn

Adding the `inline` keyword to a function definition makes that function become *semantically inlined* at the callsite. This is not a hint to be possibly observed by optimizations; rather, the body of the `inline` function is expanded at callsites during semantic analysis. This means that unlike normal function calls, comptime-known arguments of an `inline` function call become comptime-known inside the expanded body. This comptime-ness can potentially propagate all the way to the return value:

```
inline fn foo(a: i32, b: i32) i32 {
  return a + b;
}

task main() void {
  if (foo(1200, 34) != 1234) {
    @comptime_assert(false);
  }
}
```

In the code above, `foo(1200, 34)` evaluates to 1234 at comptime, so the `if` condition evaluates to false and the `@comptime_assert` is ignored. If `inline` is removed, `foo(1200, 34)` is no longer comptime-known, so the `@comptime_assert` would fail.

Since `inline` functions are expanded at callsites, they only exist in non-inlined form at comptime. As such, `inline` functions may not be used in ways that require functions to be valid at runtime; for example, `inline` functions cannot have a `linkname` and it is not allowed to take the address of an `inline` function.

`inline` functions cannot have a storage class.

It is generally better to let the compiler decide when to inline a function, except for these scenarios:

- To cause comptime-ness of the arguments to propagate to the return value of the function, as in the above example
- Real world performance measurements demand it

Note that `inline` actually *restricts* how the compiler is allowed to compile a function. This can harm binary size, compilation speed, and even runtime performance.

noinline

Adding the `noinline` keyword to a function definition prohibits that function from being inlined at callsites. It cannot be combined with the `inline` keyword or storage classes.

```
noinline fn foo(a: i32, b: i32) i32 {
  return a + b;
}
```

Direct and Indirect Function Calls

Functions can be called directly by name or indirectly through function pointers. For example:

```
fn foo(a: i16, b: f32) f32 { ... }

var foo_ptr: *const fn(i16,f32)f32 = foo;

task main() void {
  foo(42, 3.14);     // Direct function call
  foo_ptr(67, 42.0); // Indirect function call
}
```

The function value `foo` in the example above is implicitly coerced to the requested function pointer type. Note however that function values can only be coerced to `const` function pointers as shown in the example above.

It is also possible to take the address of a function symbol using the address-of operator `&` as shown in the example below:

```
fn foo() void { ... }

var foo_ptr: *const fn()void = &foo;

task main() void {
  foo_ptr(); // Indirect function call
}
```

Taking the address of a function using the `&` operator is semantically equivalent to the implicit coercion of a function value to a `const` function pointer type. This means that the resulting address will always be a `const` pointer as well.

Tasks cannot be called directly like regular functions, for example:

```
task foo() void { ... }

task invalid_foo_call() void {
  foo(); // ERROR: task cannot be called.
}
```

> **Warning**
>
> Due to a compiler limitation, it is possible to bypass this restriction by taking the address of a task and then calling the task through the respective function pointer, but this results in unspecified behavior. A future release of the compiler will disallow this.

Statements

If-expression

If-expressions have the following syntax:

```
if (condition) expr_then else expr_else
```

If-expressions can serve the role of a ternary operator:

```
const x : i32 = if (cond) 0 else 1;
```

Since blocks are expressions (see Blocks), this syntax also describes usage of `if` as a conditional statement, as opposed to an expression evaluated for its value:

```
if (condition) {
  // ...
}
else {
  // ...
}
```

If `condition` is known at compile-time, the branch not-taken is not semantically checked by the compiler, but it must still be syntactically valid. Otherwise, `expr_then` and `expr_else` must have compatible types.

The `else` clause is optional. If the `else` clause is omitted, the if-expression evaluates to a value of type `void` when its condition is false. This implies `expr_then` must have type `void` in if-expressions without an `else` clause.

It is possible to combine an `else` clause with another if-expression:

```
if (condition) {
  // ...
}
else if {
  // ...
}
else {
  // ...
}
```

For-statement

A for-statement iterates over the elements of an array or range:

```
for (my_array) |element| {
  // ...
}

for (@range(i32, 0, 2, 100)) |element| {
  // ...
}
```

Inside the loop body, the variable `element` acts as a `const` declaration whose value is the element that is currently being iterated on.

For-statements may specify a `const` declaration for the index of the element being iterated on:

```
for (my_array) |element, index| {
  // ...
}
```

A `break` statement may be used to end the loop:

```
for (my_array) |element, idx| {
  // ...
  if (condition) {
    break;
  }
}
```

A `continue` statement may be used to end the current iteration of the loop:

```
for (my_array) |element, idx| {
  // ...
  if (condition) {
    continue;
  }
}
```

When a `for` loop is labeled, it can be referenced from a `break` nested within its body. This makes it possible to `break` a loop from inner loops nested within its body:

```
outer: for (my_array) |element, idx| {
  for (other_array) |elem2, idx2| {
    // ...
    if (condition) {
      // Exit the outermost loop
      break :outer;
    }
  }
}
```

Note that, to define a label for a loop, `:` occurs *after* the name, while `:` occurs *before* the name when referring to a label in `break`.

Like identifiers, redefinition of a label is not allowed. However, labels belong to a separate namespace from identifiers. In other words, it is legal for a label to have the same name as a variable, function, or task that is in scope. Also, since labels are only visible within their corresponding loop, it is possible to reuse labels for loops that are not nested within each other.

```
foo: for (array1) |x| {
  // error: redefinition of label 'foo'
  foo: for (array2) |y| { ... }
}

fn bar() void {
  // No error
  bar: for (array3) |z| { ... }

  baz: for (array4) |a| { ... }
  // No error: not a redefinition of baz
  baz: for (array5) |b| { ... }
}
```

While-statement

While-statements have the following syntax:

```
while (condition) {
  // ...
}
```

`continue` or `break` statements may be used inside the body of a while-statement.

When a `while` loop is labeled, it can be referenced from a `break` nested within its body, including from inner loops nested within, in the same manner as a `for` loop:

```
outer: while (cond1) {
  while (cond2) {
    break :outer;
  }
}

outer: while (cond) {
  inner: for (array) |element| {
    if (cond2) {
      break :outer;
    } else if (cond3) {
      break :inner;
    }
  }
}
```

A while-statement may optionally specify an assignment expression:

```
while (condition) : (i += 3) {
  // ...
}
```

The assignment expression executes at the end of each loop iteration, including iterations finished with a `continue` statement.

Blocks

Blocks are used to limit the scope of variable declarations:

```
{
  var x: i32 = 1;
}
x += 1;
// error: use of undeclared identifier
```

A block may be labeled. When labeled, it can be referenced from a `break` nested inside, which exits the block:

```
var x: i32 = 0;
outer: {
  if (cond) {
    break :outer;
  }
  x += 1;
}
// x == 0 if cond is true
```

Note that a label is required for `break` to exit a block. In other words, `break` without a label always acts on the closest loop and a block without a label cannot be exited with `break`:

```
while (cond1) {
  blk: {
    if (cond2) {
      break; // Exits the loop
    } else {
      break :blk; // Exits 'blk'
    }
  }

  {
    if (cond3) {
      break; // Exits the loop
    }
  }
}
```

Blocks are also expressions. Labeled `break`s that refer to blocks can be used to return a value from the block:

```
var y: i32 = 123;
const x = blk: {
  y += 1;
  break :blk y;
};
@assert(x == 124);
@assert(y == 124);
```

If multiple labeled `break`s refer to a block, all of their values must have compatible type:

```
const M: i32 = 42;
const N: comptime_int = 100;
const x = blk: {
  if (...) {
    break :blk N;
  }
  break :blk M;
};
@comptime_assert(@type_of(x) == i32);

const y = blk: {
  if (...) {
    break :blk @as(i32, 1);
  }
  break :blk 0.5; // error: expected type 'i32', got: 'comptime_float'
};
```

A `break` without a value is equivalent to a `break` whose value is `void`. If control flow may reach the end of a block without breaking a value, the block's type is `void` and any values broken from the block must be `void`. By this reasoning, unlabeled blocks always have type `void` since it is not possible to break them:

```
blk: {
  if (...) {
    break :blk {}; // OK, '{}' evaluates to void
  }
}

blk: {
  if (...) {
    break :blk false; // error: expected type 'void', got: 'bool'
                      // note: block that is broken has type 'void' because
                      //       control flow may reach the end without a break
  }
}

blk: {
  if (...) {
    break :blk false; // OK, 'return' prevents control flow from reaching end
  }
  return;
}
```

A block evaluates to a comptime-known value if its type is `void` or if all of the following hold:

- The block is referred to by exactly 1 `break` with a value
- The block is guaranteed to terminate by executing this `break`
- The `break`'s value is comptime-known

A block may evaluate to a comptime-known value even if it contains runtime code:

```
const x = b: {
  runtime_code();
  break :b 1;
} + 42;
@comptime_assert(@type_of(x) == comptime_int);
@comptime_assert(x == 43);
```

Switch-statement

Switch-statements have the following syntax:

```
switch (input) {
  case_values1 => branch_expr1,
  case_values2 => branch_expr2,
  ...
  else => else_expr
}
```

`input` can be an expression of a fixed-width integer type (i.e., `comptime_int` is not allowed) or of any enum type.

The body of the switch statement consists of 1 or more comma-separated branches where each branch consists of 2 parts: the `case_values` and the corresponding `branch_expr`. A branch may combine multiple `case_value` expressions via a comma:

```
switch (input) {
  case_value1, case_value2 => branch_expr1n2,
  case_value3 => branch_expr3,
  ...
}
```

A switch statement will attempt to match `input` with one of the provided `case_value` expressions. If a match is found the corresponding branch will be selected and the respective `branch_expr` will be executed. If no match is possible, the `else` branch will be selected as the default and the corresponding `else_expr` will be executed.

`case_value` expressions must be comptime-known and coercible to the type of the `input` expression. They must also be unique.

All `branch_expr` expressions (including the `else_expr` expression, if present) must have compatible types.

If `input` is known at compile-time, the `branch_exprs` corresponding to the branches not-taken are not semantically checked by the compiler, but they must still be syntactically valid.

A switch can also be used as an expression. In this scenario all `branch_expr` expressions (including the `else_expr` expression, if present) must be able to be coerced to the common requested type:

```
fn foo(e: my_enum) i16 {
  // All branch_exprs and the else_expr are coerced to 'i16' which is the
  // type requested by the 'return' expression.
  return switch (e) {
           my_enum.A => 1,
           my_enum.B => -10,
           my_enum.C => 42,
           else => 100
         };
}
```

Branches do not fall through. If fall through behavior is desired, `case_value` expressions can be combined and if-statements can be used as follows:

```
switch (input) {
  0, 1 => {
    if (input == 0) {
      // Logic for case 0
    }
    // Common logic for cases 0 and 1
  },
  ...
}
```

A switch statement must cover all possible values for a given `input` expression type either explicitly by specifying a `case_value` for each possibility or implicitly through the `else` branch:

```
var int_input: i16 = ...;
switch (int_input) {
  -5, 0 => ...
  // ERROR: Not all possible 'i16' values are covered. An 'else' branch is
  // needed.
}

const my_enum = enum { A, B, C };
var e: my_enum = ...;
switch (e) {
  my_enum.A, my_enum.B => ...,
  my_enum.C            => ...
  // OK! No 'else' branch is needed since all possible 'my_enum' values are
  // covered.
}
```

Inline assembly expressions

> **Warning**
>
> Support for inline assembly is still experimental, and is extremely limited. Subtle pitfalls and undefined behavior are very easy to trigger with inline assembly. We suggest reading up on how inline assembly works in C before attempting to use this feature.

Inline assembly expressions have the following syntax:

```
asm volatile (             // "volatile" keyword is optional
  assembly_instructions,
  : output_constraints     // optional
  : input_constraint       // optional
  : clobbers               // optional
);
```

`assembly_instructions` is an expression of type `comptime_string` which is a templated string of the instructions to be assembled. It supports the following substitutions:

- `%[name]`: Substitute the input or output operand whose constraint is named `name`.
- `%[name:modifier]`: Substitute the input or output operand whose constraint is named `name`, with the argument modifier `modifier`.
- `%%`: Substitute a literal `%` character.
- `%=`: Substitute a decimal integer unique to this asm expression at assembly time. The compiler may duplicate asm expressions, but `%=` will be unique to this asm expression even in the presence of such duplication.

An *argument modifier* modifies how the compiler should represent the corresponding argument in the assembler output. Currently the only supported argument modifier is `a`, which indicates that the argument should be formatted as an address (e.g. the address of a global variable).

The item `output_constraints` is an optional list of comma-separated items of the form:

```
[identifier1] "constraint_string" (identifier2)
```

where:

- `identifier1` is a name used to refer to this output within the assembly instructions,
- `constraint_string` is a specifier for the form of operand that should be used for this output within the assembly instructions, and
- `identifier2` is the name of a CSL variable to which this output will be written.

The item `input_constraints` is an optional comma-separated list of the form:

```
[identifier] "constraint_string" (expr)
```

where:

- `identifier` is a name used to refer to this input within the assembly instructions,
- `constraint_string` is a specifier for the form of operand that should be used for this input within the assembly instructions, and
- `expr` is an expression that will supply the initial value for this register when the inline assembly is executed.

A valid `constraint_string` for an output constraint consists of `=`, followed optionally by `&`, and then a constraint code. The presence of `&` denotes an early-clobber output.

The `constraint_string` for an input constraint may consist of a single constraint code. Alternatively, input constraints may be a *tied constraint*. Tied constraints have the form `==[NAME]` (square brackets included). A tied constraint requires that an input be given the same register as the output named `NAME` within the assembly string. It is an error to tie multiple inputs to the same output. An input with a tied constraint must have the same type as the output it is tied to.

A constraint code may be: * of the form `{R}` (curly braces included), where `R` names a specific register that is supported on the target. The corresponding argument's bit width must match that of the named register. * `r`, specifying that the compiler should automatically select a general-purpose register operand. The corresponding argument's bit width must match that of some general-purpose register. * `m`, specifying that the compiler should select a memory address operand. The corresponding argument may not have a comptime-only type (see Comptime). * `i`, allowing comptime-known integer or comptime-known pointer values. Not valid for outputs. Integer values must be coercible to `i64`. * `n`, allowing comptime-known integer values that are coercible to `i64`. Not valid for outputs. * `X`, allowing the compiler to select any kind of operand, no constraint whatsoever. Not valid for outputs. The corresponding argument may not have a comptime-only type (see Comptime).

A constraint code may be prefixed with `*` (which goes after the `=` in an output constraint string) to denote an indirect constraint. This indicates that the assembly writes to or reads from memory at a provided address. This can be used for memory constraints, e.g. `=*m` or `*m`, to pass the address of a variable into the assembly. It is also possible to use indirect register constraints for outputs (e.g. `=*r`) but not inputs. Other constraint codes may not be indirect. Arguments corresponding to indirect constraints must have pointer type.

Supported registers are the general-purpose registers and the carry/condition flag `cflag`. On all current Cerebras architectures, the general-purpose registers are the 16-bit registers `r0` through `r15`, as well as the 32-bit double-registers `d0`, `d2`, `d4`, ..., `d14`. Note that each `dN` register is essentially aliased with `rN` and `rN+1`.

The item `clobbers` is an optional comma-separated list of clobbers, where each clobber is the name of a general-purpose register or the string `memory`. Specifying that a register is clobbered indicates to the compiler that it may be modified by the inline assembly code as a side effect, so the compiler will need to save it before entering the inline assembly block and restore it after exit. Specifying a clobber of `memory` indicates that the inline assembly may write to arbitrary memory locations.

> **Warning**
>
> Note that on some targets (namely `wse2`), `r7` is used as the stack pointer. Using `r7` in clobbers on these targets is banned. Using `r7` for input or output constraints on these targets may produce undefined behavior, and may be banned in the future.

Writing to a register without specifying it as an output or clobber register is undefined behavior. Reading from a register that is not specified in an input constraint is undefined behavior. Writing to an output register before *all* reads of inputs have occurred is undefined behavior unless that output register is specified as early-clobber.

The `volatile` keyword indicates that the assembly code has side effects not expressed in the constraints or clobbers. This means the compiler may not reorder `volatile` assembly expressions nor may it alter the amount of times a `volatile` assembly expression is executed. For example, assembly expressions marked `volatile` will not be eliminated even if nothing uses their output values or if there are no outputs.

Example

The following function uses inline assembly to return twice the value of `x+y`. (The code here is slightly inefficient, for the purposes of demonstrating inputs, outputs, and clobbers all in one go.)

```
fn doubleIt(x: i16, y: i16) i16 {
  var ret: i16;

  asm (
    @strcat("mov16 r6 = %[argval1]\n",
            "add16 r6 = r6, %[argval2]\n",
            "add16 r6 = r6, r6\n",
            "mov16 %[retval] = r6")
      : [retval] "=r" (ret)
      : [argval1] "{r2}" (x), [argval2] "==[retval]" (y)
      : "r6"
  );

  return ret;
}
```

Global Assembly

Assembly expresions that occur in top-level comptime blocks are considered *global assembly*. These differ from other assembly expressions in a few ways. First, the `volatile` keyword is not valid because all global assembly expressions are unconditionally included into the program. Second, there are no constraints or clobbers. Third, there are no template substitution rules. All global assembly strings are concatenated verbatim into one long string and assembled together.

The following code uses global assembly to define a function similar to the above example of inline assembly:

```
comptime {
  asm (
    @strcat(".global doubleIt\n",
            ".type doubleIt, @function\n",
            "doubleIt:\n",
            "  mov16 r6 = r2\n",
            "  add16 r6 = r6, r3\n",
            "  add16 r6 = r6, r6\n",
            "  mov16 r0 = r6\n",
            "  jmp r15\n",
            ".size doubleIt, . - doubleIt\n")
  );
}
```

Operators

Name	Syntax	Types	Remarks	Example
Addition	`a + b` `a += b`	Integers, floats		`2 + 5 == 7`
Subtraction	`a - b` `a -= b`	Integers, floats		`2 - 5 == -3`
Negation	`-a`	Integers, floats		`-1 == 0 - 1`
Multiplication	`a * b` `a *= b`	Integers, floats		`2 * 5 == 10`
Division	`a / b` `a /= b`	Integers, floats		`10 / 5 == 2`
Remainder of division	`a % b` `a %= b`	Integers		`10 % 3 == 1`
Bit shift left	`a << b` `a <<= b`	Integers	`b` must be unsigned.	`0b1 << 8 == 0b100000000`
Bit shift right	`a >> b` `a >>= b`	Integers	Arithmetic shift right if `a` is signed, otherwise logical shift right. `b` must be unsigned.	`0b1010 >> 1 == 0b101`
Bitwise AND	`a & b` `a &= b`	Integers		`0b011 & 0b101 == 0b001`
Bitwise OR	`a | b` `a |= b`	Integers		`0b010 | 0b100 == 0b110`
Bitwise XOR	`a ^ b` `a ^= b`	Integers		`0b011 ^ 0b101 == 0b110`
Bitwise NOT	`~a`	Fixed-width integers		`~@as(u8, 0b10101111) == 0b01010000`
Logical AND	`a and b`	`bool`	If `a` is `false`, returns `false` without evaluating `b`. Otherwise, returns `b`.	`(false and true) == false`

Logical OR	`a or b`	`bool`	If `a` is `true`, returns `true` without evaluating `b`. Otherwise, returns `b`.	`(false or true) == true`
Logical NOT	`!a`	`bool`		`!false == true`
Equality	`a == b`	Integers, floats, `bool`, `enum`, `direction`, `comptime_string`, `color`, `control_task_id`, `data_task_id`, `input_queue`, `local_task_id`, `output_queue`, `ut_id`, `type`		`(1 == 1) == true`
Inequality	`a != b`	Integers, floats, `bool`, `enum`, `direction`, `comptime_string`, `color`, `control_task_id`, `data_task_id`, `input_queue`, `local_task_id`, `output_queue`, `ut_id`, `type`		`(1 != 1) == false`
Greater than	`a > b`	Integers, floats		`(2 > 1) == true`
Greater than or equal	`a >= b`	Integers, floats		`(2 >= 1) == true`
Less than	`a < b`	Integers, floats		`(1 < 2) == true`
Less than or equal	`a <= b`	Integers, floats		`(1 <= 2) == true`

Warning

Except for logical AND and logical OR, the order in which expression operands are evaluated at runtime is undefined.

For binary operations, both operands must have exactly the same type, unless one of them is a `comptime_int` (see Comptime).

Comments

`//` begins a single-line comment. Comments beginning with `///` or `//!` are doc comments, which are only allowed in certain positions (see Doc comments). There are no multi-line comments in CSL.

```
// This function returns the value arg + 2
fn foo(arg : i16) i16 {
  var x : i16 = arg;

  // This and the next line are commented out: x will not be incremented by 1
  // x += 1;

  x += 2; // Increment x by 2

  return x;
}
```

Doc comments

Doc comments come in two forms. A *regular* doc comment begins with exactly three `/` characters (i.e., they must begin with `///` and cannot begin with `////`). A *top-level* doc comment begins with `//!`.

Regular doc comments may occur immediately before top-level functions and variable declarations, and before members of a `struct`, `enum`, and `union` type definition. Top-level doc comments may occur at the very top of a source file, and at the very top of the body of a `struct`, `enum`, or `union` declaration (i.e., just after the opening curly brace).

Doc comments are currently unused, but support for documentation generation is planned.

```
//! This file contains some nice, well-documented functions and types.
//! Please enjoy the example code.

/// Given an integer `x` of type `i16`, return `x+x`.
fn two_x(x: i16) i16 {
  return x+x;
}

/// Type for a pair of 16-bit floating point numbers.
const two_floats = struct {
  //! You can put a top-level comment here if you wish.

  /// First element of the pair.
  x: @fp16(),

  /// Second element of the pair.
  y: @fp16()
};

/// Type specifying a compass direction.
const compass_direction = enum(u16) {
  //! Again, you can put a top-level comment here if you wish.

  /// Towards the north pole.
  NORTH,

  /// If "north" is considered to be "top" and we are looking down on the
  /// globe, this is the counter-clockwise direction from our point of view.
  EAST,

  /// Opposite direction from EAST.
  WEST,

  /// Opposite direction from NORTH.
  SOUTH,
};

/// Union type containing either a 16-bit floating point number or a 16-bit
/// signed integer.
const int_or_float = union {
  //! Once again, top-level comments are allowed here.
  //! (And by the way, top-level comments can span multiple lines.)

  /// A 16-bit floating point number; the format of the number is determined
  /// by a compiler flag.
  float_val: @fp16(),

  /// A signed 16-bit integer.
  int_val: i16
};
```

Last updated on Apr 15, 2026, 7:13:14 PM.